Universidade Federal de Sergipe — Departamento de Computação
COMP0512 — Programação C · Turma T02 · 2026.2

Lista de Exercícios 2 — Recursividade, Listas, Pilhas e Filas

Conteúdo da segunda unidade: recursividade e pilha de execução, efeitos da recursão na memória, listas encadeadas simples e duplas, pilhas e filas.

Prof. André Yoshiaki Kashiwabara · `andreyoshiaki@dcomp.ufs.br`

Entrega manuscrita.

- Resolva **à mão**, em folhas avulsas. Escreva nome e matrícula em **todas** as folhas e numere-as (1/6, 2/6, . . .).
- Indique o número do exercício e a letra do item antes de cada resposta. Não é preciso copiar o enunciado.
- Código em C deve estar legível e indentado. Não é preciso escrever `#include` nem `main`, a menos que o item peça.
- Nos desenhos de memória, use uma caixa para cada nó ou variável e uma seta para cada ponteiro; escreva `NULL` explicitamente onde houver.
- Quando o item pedir "explique" ou "justifique", duas ou três frases bastam — mas a resposta sem justificativa não é considerada.

Data de entrega: _____

Salvo indicação em contrário, os exercícios usam os tipos vistos em aula:

```c
typedef struct no {            /* lista simples */
    int valor;
    struct no *prox;
} No;

typedef struct nod {           /* lista dupla */
    int valor;
    struct nod *ant, *prox;
} NoD;

typedef struct {
    NoD *ini, *fim;            /* primeiro e ultimo nos, ou NULL */
} Lista;
```

e as interfaces `pilha.h` (`pilha_cria`, `pilha_libera`, `empilha`, `desempilha`, `topo`, `vazia`) e `fila.h` (`fila_cria`, `fila_libera`, `enfileira`, `desenfileira`, `frente`, `vazia`, `tamanho`), todas com elementos `int`.

Parte I — Recursividade e pilha de execução

Exercício 1. O que acontece na ida e o que acontece na volta.

Considere a função abaixo.

```c
void h(int n) {
    if (n == 0) {
        printf("* ");
        return;
    }
    printf("%d ", n);
    h(n / 2);
```

```
    printf("%d ", n % 2);
}
```

(a) Escreva o que `h(6)` imprime e o que `h(11)` imprime.

(b) Identifique o caso base e o passo recursivo. Por que a recursão sempre termina para $n \geq 0$?

(c) Os números impressos **depois** do `*` têm um significado. Qual? Por que eles saem nessa ordem, e não na ordem inversa?

(d) Desenhe a pilha de execução no instante em que o `*` é impresso durante `h(6)`: um registro por chamada, com o valor de `n` em cada um. Quantos registros de `h` existem nesse instante?

(e) Quantos registros de `h` ficam na pilha, no máximo, durante `h(1000)`? Dê a conta, não só o número.

Exercício 2. Árvore de chamadas e o custo de recalcular.

Considere a função abaixo.

```c
int t(int n) {
    if (n <= 2)
        return n;
    return t(n - 1) + t(n - 3);
}
```

(a) Calcule `t(3)`, `t(4)`, `t(5)` e `t(6)`.

(b) Desenhe a árvore de chamadas de `t(5)`. Quantas chamadas a `t` acontecem, contando a primeira?

(c) Na árvore do item (b), quais chamadas são repetidas? Qual é a altura máxima que a pilha de execução atinge (em registros de `t`)?

(d) Escreva `int t_memo(int n, int *tab)` que guarda em `tab[i]` cada valor já calculado (`-1` indica "ainda não calculado"), e uma função `int t_rapida(int n)` que aloca `tab` no heap **uma única vez**, inicializa, chama `t_memo`, libera e devolve o resultado.

(e) Por que a tabela deve ser alocada em `t_rapida`, e não dentro de `t_memo`?

Exercício 3. Escrever funções recursivas.

Escreva as funções abaixo **sem laços**. Para cada uma, marque no código qual é o caso base e qual é o passo recursivo.

```c
int    conta_digitos(int n);              /* n >= 0; conta_digitos(0) == 1 */
int    maior(const int *v, int n);        /* n >= 1 */
int    palindromo(const char *s, int i, int j);
double potencia(double x, int n);         /* n >= 0 */
```

(a) Implemente `conta_digitos` e `maior`.

(b) Implemente `palindromo`, que devolve 1 se `s[i..j]` se lê igual nos dois sentidos e 0 caso contrário. Mostre a chamada inicial para uma string `s` inteira. Quais são os **dois** casos base?

(c) Implemente `potencia` usando $x^n = (x^{n/2})^2$ para n par e $x^n = x \cdot x^{n-1}$ para n ímpar, de modo que cada chamada faça **uma só** chamada recursiva. Quantas chamadas acontecem para $n = 16$? E se você escrevesse `potencia(x, n/2) * potencia(x, n/2)`?

(d) Um colega escreveu a função abaixo. O que acontece com `soma(6)`? E com `soma(5)`? Corrija o caso base.

```c
int soma(int n) {          /* deveria somar n + (n-2) + (n-4) + ... */
    if (n == 0)
        return 0;
    return n + soma(n - 2);
}
```

Parte II — Recursividade e memória

Exercício 4. Locais que se multiplicam pela profundidade.

Considere a função abaixo. Suponha uma pilha de execução de 8 MiB (8×2^{20} bytes) e que cada registro de ativação de `niveis` ocupe 4 KiB (4096 bytes), quase todos do vetor `linha`.

```c
void niveis(int n) {
    char linha[4000];
    if (n == 0)
        return;
    snprintf(linha, sizeof linha, "nivel %d", n);
    puts(linha);
    niveis(n - 1);
}
```

(a) Aproximadamente, qual é o maior `n` para o qual `niveis(n)` termina sem estourar a pilha? Mostre a conta.

(b) Se `linha` fosse declarado com `malloc(4000)` dentro da função (e liberado antes do `return`), esse limite mudaria? Explique o que passa a ficar na pilha e o que passa a ficar no heap.

(c) Reescreva a função para que o vetor seja alocado **uma única vez** e passado para as chamadas seguintes como parâmetro. Quem aloca e quem libera?

(d) A chamada `niveis(n - 1)` é uma chamada de cauda? Um compilador com `-O2` poderia evitar o estouro. Por que, ainda assim, você não deve contar com isso?

Exercício 5. Recursão de cauda.

Considere as quatro funções abaixo.

```c
int A(int n)          { if (n == 0) return 0;   return 1 + A(n - 1); }
int B(int n, int acc) { if (n == 0) return acc; return B(n - 1, acc + n); }
void C(int n)         { if (n == 0) return; printf("%d ", n); C(n - 1); }
int D(int n)          { if (n == 0) return 1;   return 2 * D(n - 1); }
```

(a) Diga o que `A(4)`, `B(4, 0)` e `D(4)` devolvem e o que `C(4)` imprime.

(b) Classifique cada uma como recursão de cauda ou não. Para as que não são, diga exatamente **o que fica pendente** em cada registro de ativação.

(c) Reescreva `A` e `D` em recursão de cauda, usando um parâmetro acumulador. Mostre a chamada inicial de cada uma.

(d) Reescreva `B` com um laço `while`, sem recursão. O que os parâmetros `n` e `acc` viraram?

Exercício 6. Quem recebe o bloco é dono dele.

A função abaixo devolve, num bloco do heap, a string com o caractere `c` repetido `k` vezes.

```c
char *repete(char c, int k) {
    if (k == 0) {
        char *s = malloc(1);
        s[0] = '\0';
        return s;
    }
    char *resto = repete(c, k - 1);
    char *s = malloc(k + 1);
    s[0] = c;
    strcpy(s + 1, resto);
    return s;
}
```

(a) Para `repete('x', 4)`, quantas chamadas a `malloc` acontecem e quantos bytes são alocados no total?

(b) O chamador faz `char *s = repete('x', 4); puts(s); free(s);`. Quantos blocos e quantos bytes ficam perdidos? Generalize para `k`.

(c) Corrija a função com o menor número de linhas possível. Em que ponto exato o `free` entra, e por que não pode vir antes do `strcpy`?

(d) Na versão corrigida, quantos bytes de heap ficam ocupados **ao mesmo tempo**, no máximo, durante `repete('x', k)`? Compare com a versão original.

(e) A função não confere o retorno de `malloc`. Acrescente a verificação de modo que, em caso de falha, ela devolva `NULL` sem vazar nada.

Parte III — Listas encadeadas simples

Exercício 7. Seguir as setas.

Considere o trecho abaixo, em que `cria(v)` devolve um nó novo com valor `v` e `prox` igual a `NULL`.

```c
No *a = cria(1), *b = cria(2), *c = cria(3), *d = cria(4);
a->prox = c;  c->prox = b;  b->prox = d;
No *ini = a;
                                    /* bloco 1 */
No *p = ini->prox;
ini->prox = p->prox;
p->prox = ini;
ini = p;
                                    /* bloco 2 */
No *q = ini;
while (q->prox->prox != NULL)
    q = q->prox;
q->prox->prox = ini;
ini = q->prox;
q->prox = NULL;
```

(a) Desenhe a lista (a partir de `ini`) antes do bloco 1, depois do bloco 1 e depois do bloco 2. Escreva, para cada desenho, os valores na ordem em que um percurso a partir de `ini` os visitaria.

(b) Descreva em uma frase o que o bloco 1 faz e o que o bloco 2 faz, para uma lista qualquer.

(c) Ao final do laço do bloco 2, quem é `q`? Por que a condição testa `q->prox->prox`, e não `q->prox`?

(d) O que acontece com o bloco 2 se a lista tiver um único nó? E se estiver vazia? Acrescente as proteções necessárias.

Exercício 8. Funções sobre uma lista simples.

Escreva as funções abaixo. Nenhuma delas deve imprimir nada.

```c
int  conta(const No *ini, int x);        /* quantas vezes x aparece        */
int  ordenada(const No *ini);            /* 1 se nao decrescente, 0 se nao */
No  *remove_todos(No *ini, int x);       /* tira todos os x; novo inicio   */
void tira_repetidos(No *ini);            /* lista ordenada: deixa um de cada */
```

(a) Implemente `conta` e `ordenada`. O que `ordenada` devolve para a lista vazia e para a lista de um nó? Justifique a escolha.

(b) Implemente `remove_todos`, liberando cada nó removido. Mostre que ela funciona para a lista `7 -> 7 -> 3 -> 7 -> NULL` com `x = 7`: desenhe a lista depois de cada nó removido.

(c) Implemente `tira_repetidos`: `1 -> 1 -> 2 -> 3 -> 3 -> 3 -> NULL` vira `1 -> 2 -> 3 -> NULL`. Por que ela não precisa devolver um novo início, ao contrário de `remove_todos`?

(d) Diga o custo de cada uma das quatro funções em função do número n de nós.

Exercício 9. O `insere_fim` que não insere.

Um colega escreveu a função abaixo para inserir `x` no fim da lista.

```c
void insere_fim(No *ini, int x) {
    No *novo = malloc(sizeof(No));
    novo->valor = x;
    No *p = ini;
    while (p != NULL)
        p = p->prox;
    p->prox = novo;
}
```

(a) A função tem **quatro** defeitos. Liste-os, dizendo em que linha está cada um e o que ele causa em tempo de execução.

(b) Mesmo depois de corrigir o laço, a chamada `insere_fim(lista, 5)` com `lista == NULL` não altera `lista` na `main`. Por quê?

(c) Reescreva a função de duas formas corretas: (1) `No *insere_fim(No *ini, int x)`, devolvendo o novo início, e (2) `void insere_fim2(No **pini, int x)`. Mostre a chamada de cada uma.

(d) Quanto custa inserir n valores, um a um, no fim de uma lista que começa vazia, usando essa função? O que mudaria na estrutura para que cada inserção no fim custasse um tempo constante?

Exercício 10. Ponteiro para a seta.

Considere a função abaixo e a lista `10 -> 20 -> 30 -> NULL`, guardada em `No *ini` na `main`.

```c
void remove_valor(No **pp, int x) {
    while (*pp != NULL && (*pp)->valor != x)
        pp = &(*pp)->prox;
    if (*pp != NULL) {
        No *morto = *pp;
        *pp = morto->prox;
```

```
7           free(morto);
8       }
9   }
```

(a) Mostre a chamada correta a partir da `main`.

(b) Para `x = 10`, `x = 30` e `x = 99`, diga para **que variável ou campo** `pp` aponta quando o laço termina (por exemplo: "a variável `ini` da `main`" ou "o campo `prox` do nó 20").

(c) Desenhe a memória no caso `x = 30`, logo antes do `free`: os três nós, `ini`, `pp` e `morto`.

(d) Por que essa versão não precisa de um `if` separado para o primeiro nó, ao contrário da versão com `ant`?

(e) O que aconteceria se as duas condições do `while` fossem escritas na ordem inversa? Para qual dos três valores de `x` o problema aparece?

Parte IV — Listas duplamente encadeadas

Exercício 11. Quatro setas para mover um nó.

A lista `l` contém `10 <-> 20 <-> 30 <-> 40`, com `l->ini` no 10 e `l->fim` no 40. O ponteiro `p` aponta para o nó 20 e `q` para o nó 40.

```
1  p->ant->prox = p->prox;      /* 1 */
2  p->prox->ant = p->ant;       /* 2 */
3  p->ant  = q;                 /* 3 */
4  p->prox = NULL;              /* 4 */
5  q->prox = p;                 /* 5 */
6  l->fim  = p;                 /* 6 */
```

(a) Desenhe a lista depois da linha 2 e depois da linha 6, com todas as setas `ant` e `prox`, `l->ini` e `l->fim`.

(b) Escreva o que um percurso de `l->ini` pelos `prox` imprime e o que um percurso de `l->fim` pelos `ant` imprime, ao final.

(c) Diga o valor de `l->ini->prox->prox->ant->valor` ao final.

(d) Se `p` apontasse para o nó 10 (o primeiro), que linha falharia, e por quê? Reescreva as linhas 1 e 2 para funcionarem também nesse caso.

(e) E se `p` e `q` apontassem ambos para o 40? Siga as linhas na ordem e diga o que acontece.

Exercício 12. Operações nos extremos e no meio.

Escreva as funções abaixo para a lista dupla com `ini` e `fim`.

```
1  void insere_depois(Lista *l, NoD *p, int x);  /* p pertence a l */
2  int  remove_ultimo(Lista *l);                 /* devolve o valor; lista nao vazia */
3  void imprime_reversa(const Lista *l);
```

(a) Implemente `insere_depois`. Quantos ponteiros mudam quando `p` está no meio? E quando `p == l->fim`?

(b) Implemente `remove_ultimo` sem laço. Não esqueça o caso em que a lista tem um único nó: o que deve acontecer com `ini` e `fim`?

(c) Implemente `imprime_reversa` sem recursão e sem estrutura auxiliar.

(d) Remover o último nó de uma lista **simples** com `ini` e `fim` custa quanto? Por que manter o ponteiro `fim` não resolve o problema na lista simples, mas resolve na dupla?

Exercício 13. O nó sentinela.

Uma lista dupla circular com nó cabeça `cab` (que não guarda valor) tem `cab->prox` no primeiro nó e `cab->ant` no último; a lista vazia tem `cab->ant == cab->prox == cab`. A inserção é:

```c
void insere_antes(NoD *p, int x) {
    NoD *novo = malloc(sizeof *novo);
    novo->valor = x;
    novo->prox = p;
    novo->ant  = p->ant;
    p->ant->prox = novo;
    p->ant = novo;
}
```

(a) Desenhe a lista vazia. Depois, desenhe a lista após a chamada `insere_antes(cab, 5)` e, em seguida, após `insere_antes(cab, 7)`.

(b) As duas chamadas do item (a) inseriram no início ou no fim? Que chamada insere no outro extremo?

(c) Escreva `int tamanho(const NoD *cab)`. Qual é a condição de parada do percurso, e por que não pode ser `p != NULL`?

(d) Escreva `void remove_no(NoD *p)` em três linhas, sem nenhum `if`. Explique por que a versão sem sentinela precisava de dois `if`.

(e) O que acontece se alguém chamar `remove_no(cab)`? Como a função poderia se proteger, se recebesse também `cab`?

Parte V — Pilhas

Exercício 14. Seguir a pilha.

Responda usando apenas a definição de pilha (o último a entrar é o primeiro a sair).

(a) Avalie a expressão pós-fixa `5 1 2 + 4 * + 3 -` com uma pilha: número empilha; operador desempilha dois valores e empilha o resultado. Desenhe a pilha depois de cada símbolo e dê o resultado final.

(b) No item (a), quando o operador `-` desempilha dois valores, qual deles é o operando da esquerda? Mostre o que sairia se a ordem fosse trocada.

(c) Os valores `1 2 3 4` são empilhados nessa ordem, mas a qualquer momento pode-se desempilhar e imprimir o topo. Para cada sequência abaixo, diga se ela pode ser impressa; se puder, dê a sequência de `empilha` e `desempilha`; se não puder, diga em que momento ela trava.

`2 4 3 1` `3 1 4 2` `4 3 2 1` `1 4 2 3`

Exercício 15. Pilha num vetor que dobra.

Um colega implementou a pilha num vetor com a estrutura abaixo.

```c
struct pilha {
    int *v;
    int n;      /* quantos elementos ha; o topo e v[n-1] */
    int cap;    /* quantos cabem em v */
};

void empilha(Pilha *p, int x) {
    if (p->n == p->cap) {
        p->cap = 2 * p->cap;
        p->v = realloc(p->v, p->cap * sizeof *p->v);
```

```c
    }
    p->v[p->n++] = x;
}

int desempilha(Pilha *p) {
    return p->v[p->n--];
}

int topo(const Pilha *p) {
    return p->v[p->n];
}
```

(a) `desempilha` e `topo` têm o mesmo erro. Qual é? Com a pilha contendo `10 20 30` (30 no topo), o que cada uma devolveria?

(b) Nenhuma das duas trata a pilha vazia. O que acontece se `desempilha` for chamada com `n == 0`, já com o erro do item (a) corrigido?

(c) `empilha` tem dois problemas no `realloc`: um quando ele falha e outro quando a capacidade inicial é 0. Explique ambos e reescreva a função.

(d) Começando com `cap = 1`, empilham-se 1000 valores. Quantas vezes o vetor cresce? Quantos elementos são copiados no total pelos `realloc` (no pior caso, em que cada um muda o bloco de lugar)? E se, em vez de dobrar, a capacidade aumentasse de 1 em 1?

Exercício 16. Pilha numa lista e o uso só pela interface.

(a) Escreva `struct pilha`, `empilha`, `desempilha` e `pilha_libera` para uma pilha guardada numa lista simples (use o tipo `No`).

(b) O topo deve ficar no **primeiro** ou no **último** nó? Justifique com o custo de `empilha` e `desempilha` em cada escolha.

(c) Usando **apenas** as funções de `pilha.h` (sem acessar os campos), escreva `int base(Pilha *p)`, que devolve o elemento do fundo de uma pilha não vazia e deixa a pilha exatamente como estava. De que estrutura auxiliar você precisou?

(d) A função do item (c) funciona sem nenhuma alteração tanto com a pilha num vetor quanto com a pilha numa lista. Que característica de `pilha.h` garante isso?

Parte VI — Filas

Exercício 17. A fila circular dá a volta.

Uma fila circular guarda `v`, `ini` (posição da frente), `n` (quantos há) e `cap`. Ela começa vazia, com `cap = 5` e `ini = 0`; o valor novo entra em `v[(ini + n) % cap]`.

```c
enfileira(f, 10);  enfileira(f, 20);  enfileira(f, 30);  enfileira(f, 40);
desenfileira(f);   desenfileira(f);   desenfileira(f);
enfileira(f, 50);  enfileira(f, 60);  enfileira(f, 70);
```

(a) Desenhe `v[0..4]` e dê `ini` e `n` depois da primeira linha, depois da segunda e depois da terceira. Marque as posições que guardam valores que já não pertencem à fila.

(b) Em que posição entra o próximo valor? Escreva a conta.

(c) Agora `enfileira(f, 80)` e `enfileira(f, 90)`. O 90 não cabe e a fila cresce para `cap = 10`, copiando os elementos na ordem da fila para o início do vetor novo. Desenhe `v[0..9]`, `ini` e `n` depois do 90.

(d) Por que não basta fazer `v = realloc(v, 10 * sizeof *v)`, como na pilha? Em que ordem os valores sairiam nesse caso?

(e) Uma versão alternativa guarda `ini` e `fim` (próxima posição livre) em vez de `ini` e `n`. Depois do `enfileira(f, 80)`, quanto valem `ini` e `fim`? Por que, com só esses dois campos, não dá para distinguir a fila cheia da fila vazia?

Exercício 18. Fila numa lista: as duas pontas.

Um colega implementou a fila numa lista simples com `struct fila { No *ini, *fim; int n; };`.

```c
void enfileira(Fila *f, int x) {
    No *novo = malloc(sizeof *novo);
    novo->valor = x;
    novo->prox = NULL;
    f->fim->prox = novo;
    f->fim = novo;
    f->n++;
}

int desenfileira(Fila *f) {
    No *velho = f->ini;
    int x = velho->valor;
    f->ini = velho->prox;
    free(velho);
    f->n--;
    return x;
}
```

(a) O que acontece na primeira chamada de `enfileira` sobre uma fila recém criada, com `ini` e `fim` iguais a `NULL`? Corrija a função testando se `f->fim` é `NULL`.

(b) Com `enfileira` corrigida, faça estas três chamadas, nesta ordem:
`enfileira(f, 1);  desenfileira(f);  enfileira(f, 2);`
Desenhe `ini`, `fim` e os nós depois de cada uma. Onde está o defeito de `desenfileira` e qual categoria de erro de memória ele produz?

(c) Corrija `desenfileira`, incluindo o tratamento da fila vazia.

(d) Por que a fila sai pelo **primeiro** nó e entra depois do **último**, e não o contrário? Diga o custo de cada operação nas duas escolhas.

Exercício 19. Resolver problemas com filas e pilhas.

(a) Na brincadeira da batata quente, n crianças numeradas de 1 a n estão numa fila; a cada rodada, a da frente passa para o fim k vezes e, em seguida, a que ficou na frente sai. Para $n = 6$ e $k = 2$, desenhe a fila no início de cada rodada e dê a ordem em que as crianças saem.

(b) Usando apenas `fila.h`, escreva `void intercala(Fila *a, Fila *b, Fila *saida)`, que tira alternadamente um elemento de `a` e um de `b` e os coloca em `saida`; quando uma acaba, o resto da outra vai inteiro.

(c) Usando uma fila e uma pilha (um `char` cabe num `int`), escreva `int eh_palindromo(const char *s)`. Explique por que as duas estruturas, juntas, resolvem o problema.

(d) `pilha.h` e `fila.h`, como estão, não podem ser incluídos no mesmo arquivo `.c`. Por quê? Proponha uma correção nas duas interfaces.

Exercício 20. Escolher a estrutura.

(a) Copie e preencha a tabela com o custo de cada operação ($O(1)$ ou $O(n)$) numa coleção de n elementos.

Operação	Vetor	Lista simples (`ini`)	Lista simples (`ini`, `fim`)	Lista dupla (`ini`, `fim`)
inserir no início				
inserir no fim				
remover do início				
remover do fim				
acessar o k-ésimo				
buscar um valor				

(b) Na coluna "Vetor", considere que "inserir no fim" usa um vetor que dobra quando enche. Por que se diz que essa operação custa $O(1)$ *amortizado*, se de vez em quando ela copia o vetor inteiro?

(c) Para cada situação abaixo, diga se você usaria uma pilha, uma fila, uma lista simples ou uma lista dupla, e justifique em uma frase:

(i) o "desfazer" de um editor de texto;
(ii) os documentos esperando numa impressora compartilhada;
(iii) uma *playlist* com os botões "anterior" e "próxima" e remoção da música atual;
(iv) verificar se as `{ }` de um programa estão balanceadas;
(v) as senhas de atendimento de uma agência bancária.

(d) Para qual das situações do item (c) mais de uma estrutura serve bem? Compare as duas opções.